ImmersionTrack System Summary

Project Overview
- ImmersionTrack is a complete Grade 12 work immersion management platform built with Django.
- It supports student onboarding, company placement, task submission, attendance tracking, journal logging, evaluation, a

Main Objectives
- Provide a secure role-based portal for students, supervisors, and dean/admin staff.
- Streamline company assignment and student registration.
- Enable task publishing, submission, and grading.
- Track daily attendance and immersion hours.
- Collect journal entries and reflection logs.
- Support rubric-based student evaluation.
- Offer audit logs, notifications, and administrative reports.

Core Modules
- accounts: custom user model, login, registration, approval workflow.
- students: student profiles, requirements, school-year and section data.
- companies: company profiles, immersion groups, join codes.
- supervisors: supervisor profiles and management.
- tasks: task creation, submission, grading, file uploads.
- attendance: daily attendance records, approval workflow.
- journals: daily logbook entries and supervisor review.
- evaluations: rubric-based evaluation and scoring.
- announcements: school and group announcements.
- notifications: in-app notification count and alerts.
- auditlogs: activity logging and system audit trail.
- dean: dean/admin portal for approval and monitoring.
- core: role-based redirects and shared utilities.

User Roles & Responsibilities
- Student: register, join companies, submit tasks, log attendance, write journals, view evaluations, and receive announcem
- Supervisor: manage students, create tasks, grade submissions, approve attendance, review journals, and evaluate stude
- Dean / Administrator: approve registrations, create companies and supervisors, monitor progress, post announcements, v

Technical Architecture
- Backend: Django framework with custom user model and role-based access.
- Frontend: Django templates with Bootstrap 5 and Crispy Forms.
- Database: SQLite for development; PostgreSQL recommended for production.
- Admin UI: Jazzmin professional admin interface.
- Media: file uploads stored in media/. Static files served from static/ and staticfiles/.
- Email: console backend for development; SMTP supported for production.

Deployment Notes
- Use Python virtual environment and install dependencies from requirements.txt.
- Run migrations and create demo or admin accounts.
- Set DEBUG = False, change SECRET_KEY, and configure ALLOWED_HOSTS in production.
- Use PostgreSQL and SMTP for live deployment.
- Run collectstatic and serve static files with a web server.

Suggested PPT Structure